

HDOC DAY-35 ASSIGNMENT

Operations on a Variable-Length Square Matrix

1. Problem Understanding

The task is to write a menu-driven C program that reads a variable-length square integer matrix and performs the required operations using separate functions.

- Check whether all elements on the main diagonal are distinct.
- Find the sum of all main diagonal elements.
- Display the matrix elements using diagonal traversal, beginning at the top-left corner and traversing successive diagonals alternately in the upward-right and downward-left directions.
- The user enters the order n of the square matrix and then enters all $n \times n$ matrix elements. The menu continues until Exit is selected.

2. Standard Test Cases

The test cases include all required inputs: menu choice, matrix order n , and every matrix element.

TC	Choice	n	Matrix Elements (row-wise)	Expected Result
TC1	1	3	1 2 3 / 4 5 6 / 7 8 9	Main diagonal: 1 5 9; all distinct
TC2	1	3	1 2 3 / 4 1 6 / 7 8 9	Main diagonal: 1 1 9; not distinct
TC3	2	3	1 2 3 / 4 5 6 / 7 8 9	Main diagonal sum = 15
TC4	3	3	1 2 3 / 4 5 6 / 7 8 9	Traversal: 1 2 4 7 5 3 6 8 9
TC5	4	3	1 2 3 / 4 5 6 / 7 8 9	Program exits

3. Algorithm

A. Common Menu Logic

1. Start.
2. Read the order n of the square matrix.
3. Declare a variable-length matrix $A[n][n]$.
4. Read all $n \times n$ elements of the matrix.
5. Display the menu options.
6. Read the user choice.
7. Use switch-case and call the corresponding function.
8. Repeat the menu until the user selects Exit.
9. Stop.

B. Check Distinct Main Diagonal Elements

1. Compare every main diagonal element $A[i][i]$ with the diagonal elements after it.
2. If any two diagonal elements are equal, mark the diagonal as not distinct.
3. Display the main diagonal elements.
4. Display whether all main diagonal elements are distinct.

C. Sum of Main Diagonal Elements

1. Initialize sum = 0.
2. For i from 0 to n-1, add A[i][i] to sum.
3. Display the sum.

D. Diagonal Traversal

1. Consider each diagonal using $s = \text{row} + \text{column}$, from $s = 0$ to $2n-2$.
2. For even-numbered diagonals, traverse in the upward-right direction.
3. For odd-numbered diagonals, traverse in the downward-left direction.
4. Display every valid matrix element encountered.

4. C Program

```
#include <stdio.h>

void checkDistinct(int n, int a[n][n])
{
    int i, j, flag = 1;
    printf("Main diagonal elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i][i]);
    printf("\n");
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i][i] == a[j][j])
                flag = 0;
        }
    }
    if (flag == 1)
        printf("All main diagonal elements are distinct.\n");
    else
        printf("Main diagonal elements are not distinct.\n");
}

void diagonalSum(int n, int a[n][n])
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum = sum + a[i][i];
}
```

```

    printf("Sum of main diagonal elements = %d\n", sum);
}
void diagonalTraversal(int n, int a[n][n])
{
    int s, i, j;
    printf("Diagonal traversal: ");
    for (s = 0; s <= 2 * n - 2; s++)
    {
        if (s % 2 == 0) {
            i = s < n ? s : n - 1;
            j = s - i;
            while (i >= 0 && j < n)
            {
                printf("%d ", a[i][j]);
                i--;
                j++;
            }
        }
        else
        {
            j = s < n ? s : n - 1;
            i = s - j;
            while (j >= 0 && i < n)
            {
                printf("%d ", a[i][j]);
                i++;
                j--;
            }
        }
    }
    printf("\n");
}
int main()
{

```

```
int n, i, j, choice;
printf("Enter the order of square matrix: ");
scanf("%d", &n);
int a[n][n];
printf("Enter matrix elements:\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++)
        scanf("%d", &a[i][j]);
}
do
{
    printf("\nMENU\n");
    printf("1. Check distinct main diagonal elements\n");
    printf("2. Sum of main diagonal elements\n");
    printf("3. Diagonal traversal\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            checkDistinct(n, a);
            break;
        case 2:
            diagonalSum(n, a);
            break;
        case 3:
            diagonalTraversal(n, a);
            break;
        case 4:
            printf("Exiting program.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
}
```

```

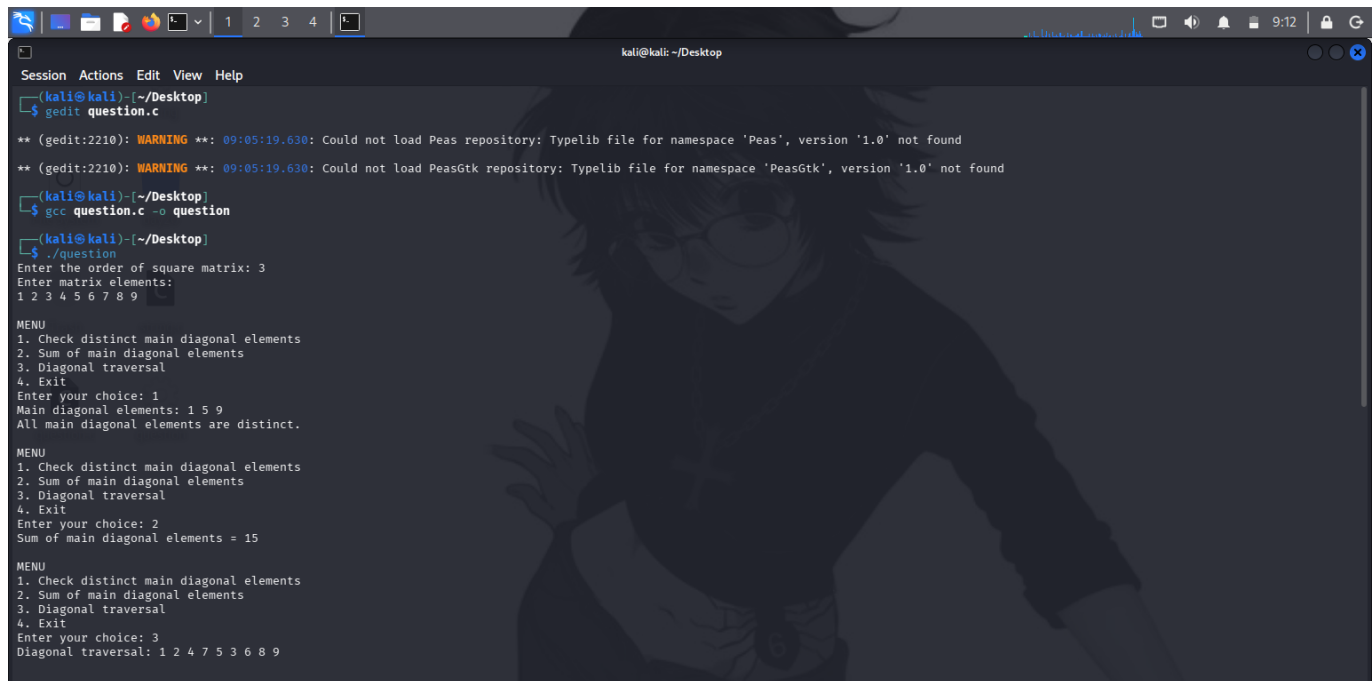
    }
}

while (choice != 4);

return 0;
}

```

5. Compilation and Execution Evidence



```

kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
└─$ gedit question.c

** (gedit:2210): WARNING **: 09:05:19.630: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2210): WARNING **: 09:05:19.630: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~/Desktop]
└─$ gcc question.c -o question

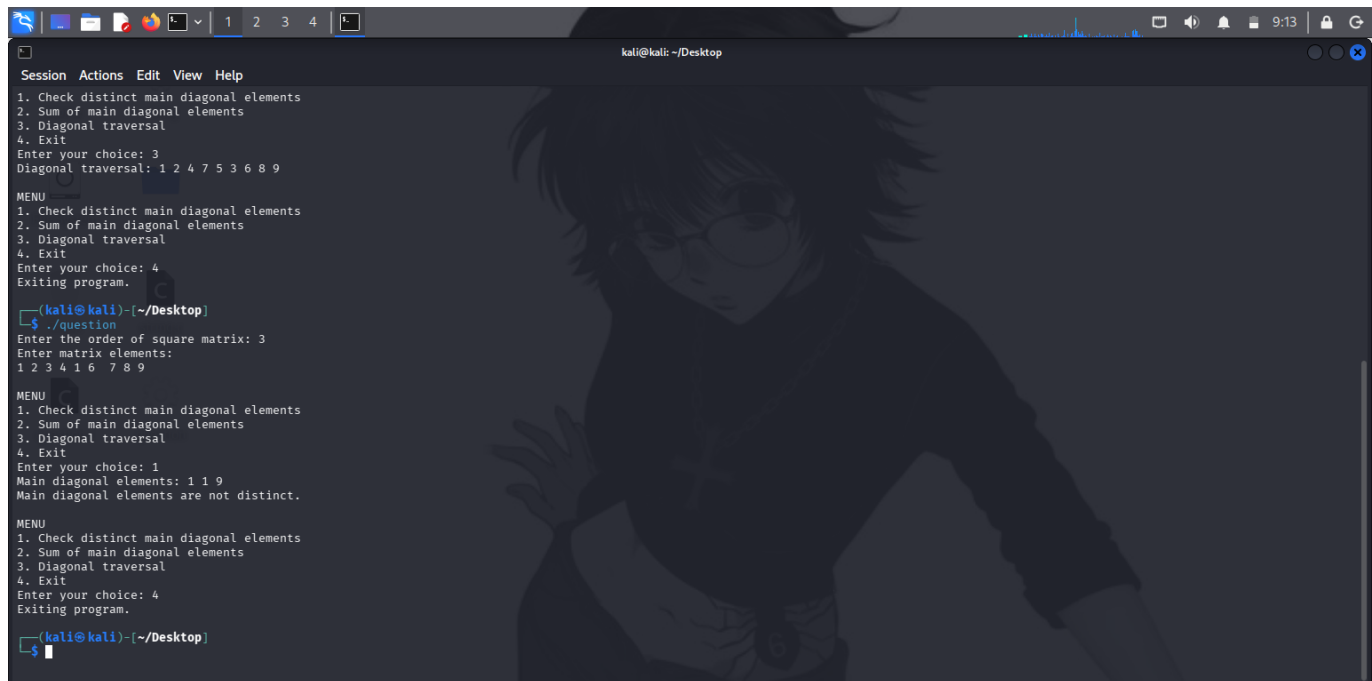
(kali@kali)-[~/Desktop]
└─$ ./question
Enter the order of square matrix: 3
Enter matrix elements:
1 2 3 4 5 6 7 8 9

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All main diagonal elements are distinct.

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements = 15

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

```



```

kali@kali: ~/Desktop
Session Actions Edit View Help
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program.

(kali@kali)-[~/Desktop]
└─$ ./question
Enter the order of square matrix: 3
Enter matrix elements:
1 2 3 4 1 6 7 8 9

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 1 9
Main diagonal elements are not distinct.

MENU
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program.

(kali@kali)-[~/Desktop]
└─$

```

6. Verification of All Test Cases

TC	Choice	n	Input Matrix	Actual Result	Status
TC1	1	3	1 2 3 / 4 5 6 / 7 8 9	Diagonal 1 5 9; distinct	Pass
TC2	1	3	1 2 3 / 4 1 6 / 7 8 9	Diagonal 1 1 9; not distinct	Pass
TC3	2	3	1 2 3 / 4 5 6 / 7 8 9	Sum = 15	Pass
TC4	3	3	1 2 3 / 4 5 6 / 7 8 9	1 2 4 7 5 3 6 8 9	Pass
TC5	4	3	1 2 3 / 4 5 6 / 7 8 9	Program exits	Pass